МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

УНИВЕРСИТЕТ ИТМО

Факультет ПИиКТ

Лабораторная работа №3

ASP.NET MVC

по дисциплине «Создание программного обеспечения»

Выполнил: студент Луценко Владимир
Группа: К3420
Преподаватель: Осипов Н. А.

Санкт-Петербург, 2026

Содержание

1 Введение **3**

2 ЛР 5. Разработка модели **3**

3 ЛР 6. Scaffold-контроллеры и аннотации **4**

4 ЛР 7. AJAX и частичные представления **5**

5 ЛР 8. Авторизация и аутентификация **6**

6 Заключение **7**

1. Введение

Цель работы – освоение технологии ASP.NET Core MVC. В рамках лабораторных работ 5–8 разработано приложение MvcCreditApp для управления кредитными продуктами: создана модель данных, реализованы CRUD-контроллеры, добавлен AJAX-поиск и настроена авторизация через Identity.

2. ЛР 5. Разработка модели

Создан проект ASP.NET Core MVC с шаблоном Individual User Accounts и базой данных SQLite. Определены модели `Credit` (кредитный продукт) и `Bid` (заявка). Контекст `CreditContext` содержит `DbSet` для обеих сущностей.

Listing 1: Модель Credit.cs

```
public class Credit
{
    public int CreditId { get; set; }
    [DisplayName("Название")]
    [Required]
    public string Head { get; set; }
    [DisplayName("Срок мес(.)")]
    public int Period { get; set; }
    [DisplayName("Сумма")]
    public decimal Sum { get; set; }
    [DisplayName("Процент")]
    public double Procent { get; set; }
}
```

Класс `SeedData` добавляет 5 тестовых кредитов при первом запуске. В `HomeController` выводится таблица кредитов и форма подачи заявки.

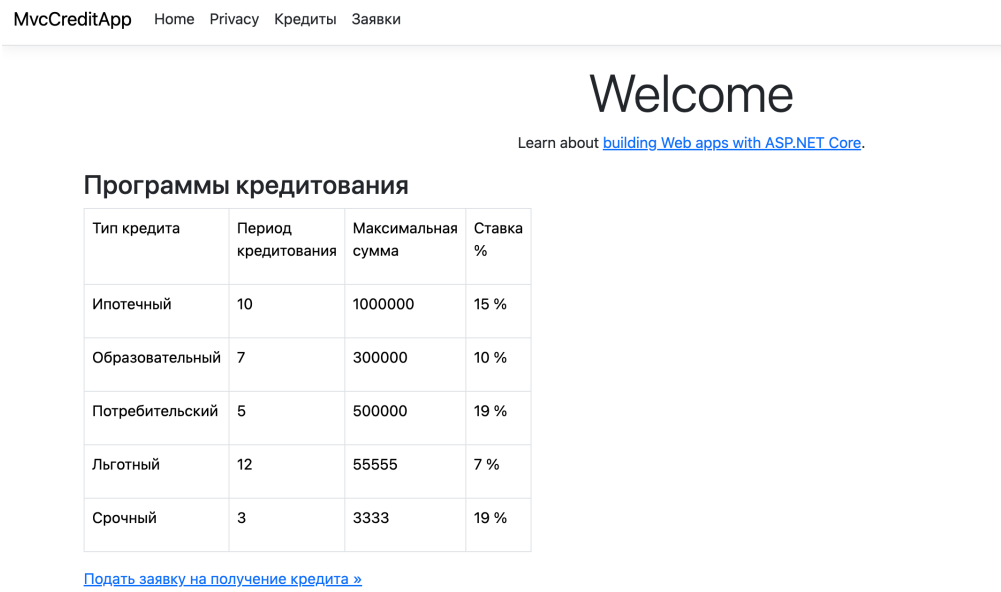

Рисунок 1 – Главная страница с таблицей кредитов

На главной странице отображается таблица «Программы кредитования» с колонками: Тип кредита, Период, Максимальная сумма, Ставка. Внизу – ссылка «Подать заявку на получение кредита».

3. ЛР 6. Scaffold-контроллеры и аннотации

Через scaffolding сгенерированы `CreditsController` и `BidsController` с CRUD-действиями. Для каждого – 5 представлений (Index, Details, Create, Edit, Delete), итого 10. К моделям добавлены аннотации `[DisplayName]` и `[Required]`. На Index кредитов установлен `[ResponseCache]`.

Listing 2: Метод CreateBid в HomeController

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CreateBid(Bid bid)
{
    if (ModelState.IsValid)
    {
        bid.bidDate = DateTime.Now;
        _context.Bids.Add(bid);
        await _context.SaveChangesAsync();
        return RedirectToAction("Index");
    }
```

```
    return View("Index");
}
```

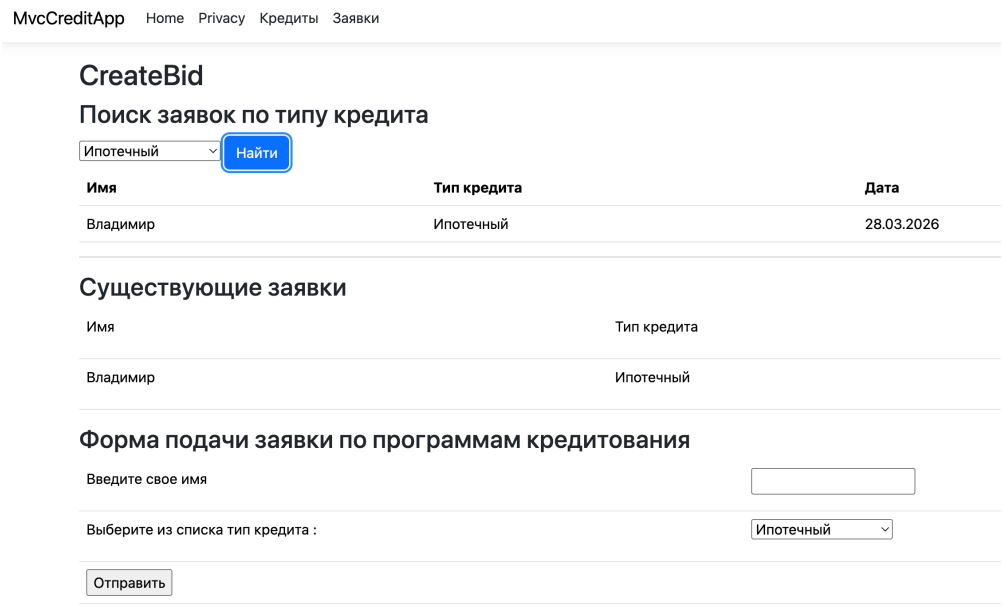

Рисунок 2 – Страница CreateBid с AJAX-поиском заявок

На странице `/Credits` (доступна только администратору) отображается список кредитов с возможностью создания, редактирования и удаления записей.

4. ЛР 7. AJAX и частичные представления

В `HomeController` добавлен метод `BidSearch` для поиска заявок по типу кредита. Результат возвращается через частичное представление `_BidSearchPartial.cshtml`. Вместо `Ajax.BeginForm` используется `fetch()` API.

Listing 3: AJAX-поиск заявок

```
public IActionResult BidSearch(string creditHead)
{
    var bids = _context.Bids
        .Where(b => b.CreditHead == creditHead)
        .ToList();
    return PartialView("_BidSearchPartial", bids);
}
```

Listing 4: Скрипт fetch() на стороне клиента

```
<script>
document.getElementById("searchBtn")
  .addEventListener("click", function() {
    var type = document.getElementById("creditType").value;
    fetch("/Home/BidSearch?creditHead=" + type)
      .then(r => r.text())
      .then(html => {
        document.getElementById("results").innerHTML = html;
      });
});
</script>
```

На странице `/Home/CreateBid` реализован поиск заявок: выбор типа кредита из выпадающего списка, кнопка «Найти». Результат подгружается без перезагрузки страницы через AJAX.

5. ЛР 8. Авторизация и аутентификация

Настроена авторизация через ASP.NET Core Identity с ролями `admin` и `user`. При инициализации создаётся администратор (`admin@credit.com`).

Listing 5: Инициализация ролей в SeedData

```
public static async Task InitializeRoles(
    IServiceProvider serviceProvider)
{
    var roleManager = serviceProvider
        .GetRequiredService<RoleManager<IdentityRole>>();
    string[] roles = { "admin", "user" };
    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
            await roleManager.CreateAsync(
                new IdentityRole(role));
    }
}
```

На метод `CreateBid` установлен атрибут `[Authorize]`, требующий авторизации для подачи заявки. CRUD-операции с кредитами и удаление

заявок доступны только администраторам через `[Authorize(Roles = "admin")]`.

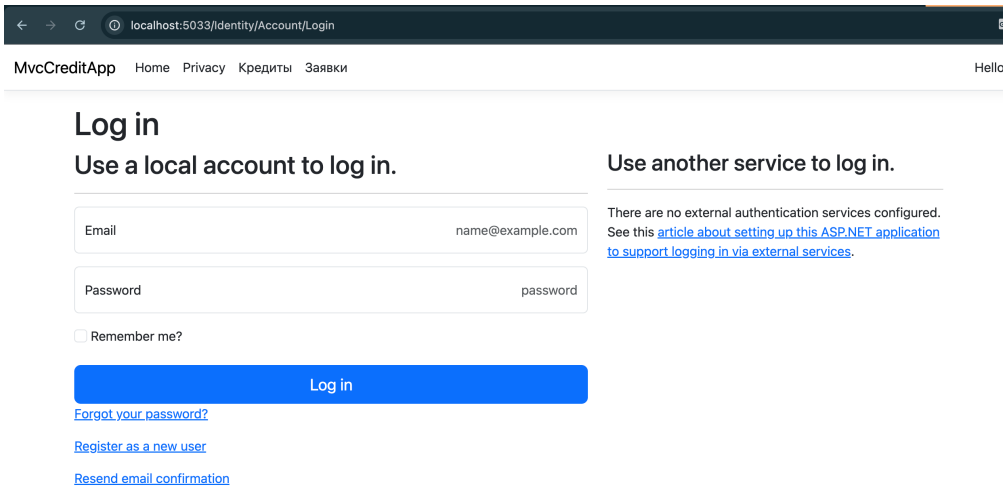

Рисунок 3 – Страница входа Identity

Страница входа `/Identity/Account/Login` содержит форму с полями Email и Password, а также ссылки на регистрацию и восстановление пароля.

6. Заключение

В ходе лабораторных работ 5–8 разработано MVC-приложение для управления кредитными продуктами. Реализованы модель данных с Entity Framework Core, CRUD-контроллеры через scaffolding, AJAX-поиск с частичными представлениями и система авторизации с разделением ролей. Приложение работает на SQLite и поддерживает регистрацию, вход и разграничение доступа.